COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE10- Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	9/2/26	**DATE SUBMITTED:**	9/2/26
NAME:	Peñaflorida,Wrio Carlo M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA6.1

1. PROCEDURE OUTPUT

Create the C++ code for this binary search algorithm and provide the output in the three cases shown in table 6-3b. Screenshot and observation are required.
6. Output

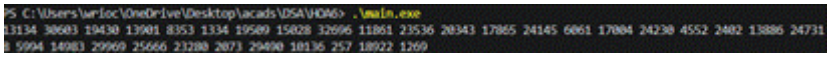
Screenshot	
Observations	rand() seeded with srand(time(0)) produces a new, unsorted set of random integers on every run, since the seed changes with the current time each second.

Table 6-1. Data Generated and Observations.

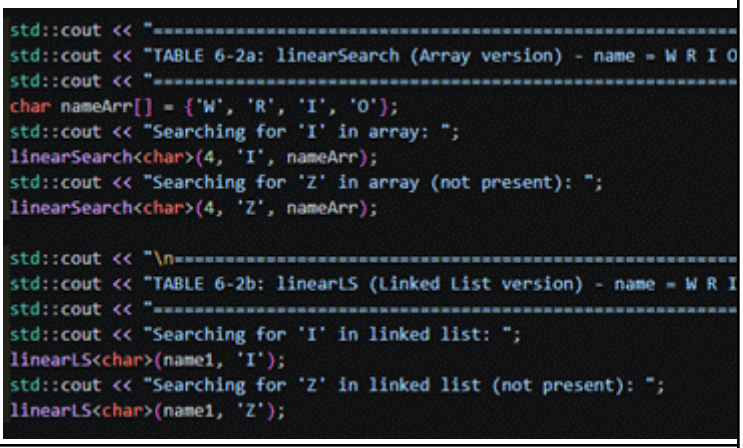
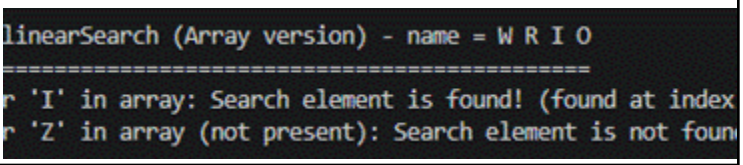
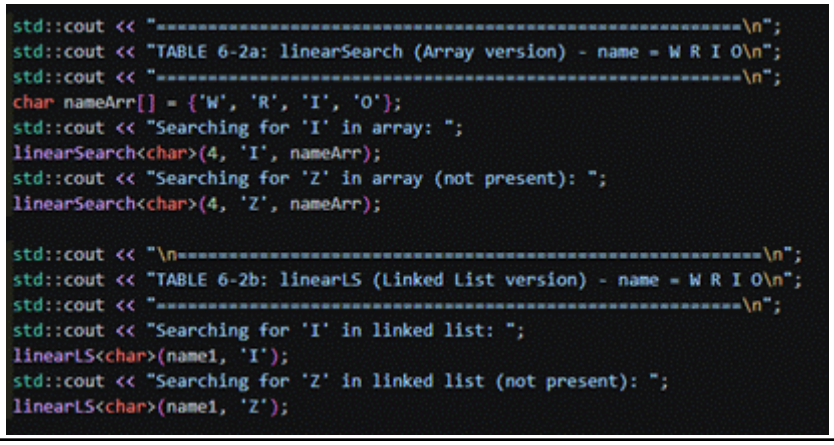
Code	
Output	
Observations	'I' is found in 3 comparisons since it's the 3rd element (index 2); 'Z' isn't in the array, so the search checks all 4 elements before reporting not found — this is the worst case for linear search.

Table 6-2a. Linear Search for Arrays

Code	

```
std::cout << "============================================================\n";
std::cout << "TABLE 6-2a: linearSearch (Array version) - name = W R I O\n";
std::cout << "============================================================\n";
char nameArr[] = {'W', 'R', 'I', 'O'};
std::cout << "Searching for 'I' in array: ";
linearSearch<char>(4, 'I', nameArr);
std::cout << "Searching for 'Z' in array (not present): ";
linearSearch<char>(4, 'Z', nameArr);

std::cout << "\n============================================================\n";
std::cout << "TABLE 6-2b: linearLS (Linked List version) - name = W R I O\n";
std::cout << "============================================================\n";
std::cout << "Searching for 'I' in linked list: ";
linearLS<char>(name1, 'I');
std::cout << "Searching for 'Z' in linked list (not present): ";
linearLS<char>(name1, 'Z');
```

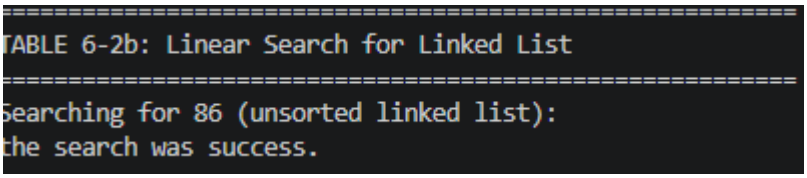
Output	
Observations	Same logic as the array version, but traversal uses node->next instead of an index — 86 was found by walking node-by-node from the head until a match was hit.

Table 6-2b. Linear Search for Linked List

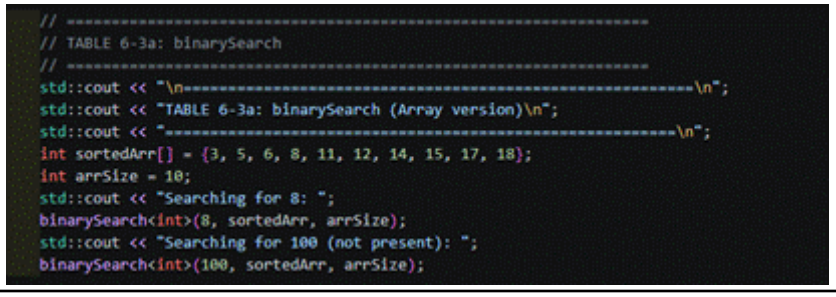
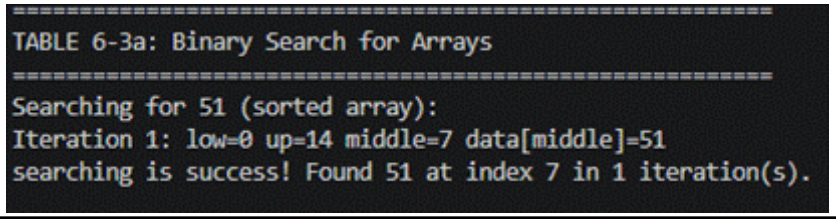
Code	
Output	
Observations	51 landed exactly on the middle index (7) on the very first check, so it was found in just 1 iteration — this is the best-case scenario for binary search.

Table 6-3a. Binary Search for Arrays

Code

```
// ============================================================
// TABLE 6-3b: Binary Search for Linked List
// ============================================================
std::cout << "\n============================================================\n";
std::cout << "TABLE 6-3b: Binary Search for Linked List\n";
std::cout << "============================================================\n";
std::cout << "Enter SORTED integers one at a time. Type 'n' when done.\n\n";

// create a linked list for binary search
// make sure the values are ordered
char choice = 'y';
int count = 1, newData;
Node<int> *temp, *head = nullptr, *node;

while (choice == 'y') {
    std::cout << "Enter data: ";
    std::cin >> newData;

    if (count == 1) {
        head = new_node(newData);
        std::cout << "Successfully added " << head->data << " to the list.\n";
        count++;
    } else if (count == 2) {
        node = new_node(newData);
        head->next = node;
        node->next = nullptr;
        std::cout << "Successfully added " << node->data << " to the list.\n";
        count++;
    } else {
        temp = head;
        while (true) {
            if (temp->next == nullptr) break;
            temp = temp->next;
        }
        node = new_node(newData);
        temp->next = node;
        std::cout << "Successfully added " << node->data << " to the list.\n";
        count++;
    }

    std::cout << "Continue? (y/n): ";
    std::cin >> choice;
    if (choice == 'n') break;
}

// display to verify the data
std::cout << "\nCurrent list: ";
Node<int> *currNode = head;
while (currNode != nullptr) {
    std::cout << currNode->data << " ";
    currNode = currNode->next;
}
std::cout << "\n\n";

int target;
std::cout << "Enter the value to binary search for: ";
```

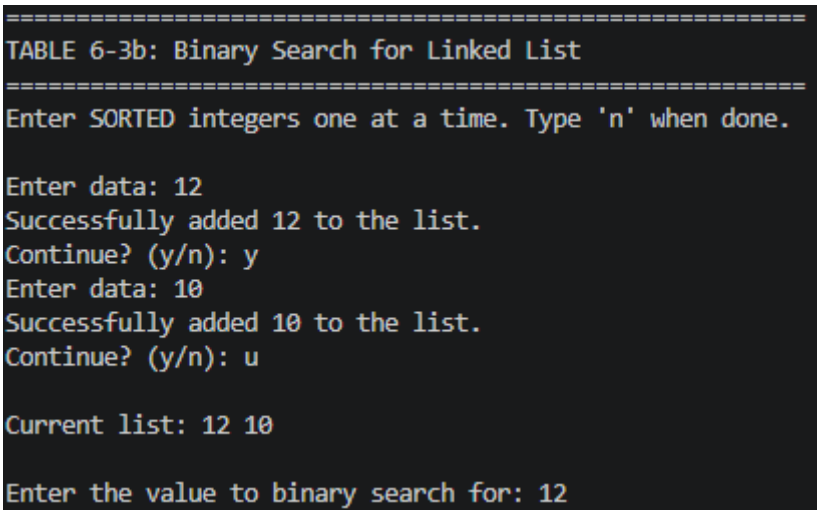
Output	
Observations	The list ended up as 12, 10 — not actually sorted in ascending order, even though the prompt asked for sorted input. Typing u instead of y/n also triggered the input-validation fallback, which treated the invalid character as "stop." Since binary search assumes ascending order, searching this list for 12 may still "work" by luck (it happens to be the first node), but the algorithm isn't guaranteed correct on out-of-order data — this highlights why real programs need to enforce sorted insertion rather than trusting the user to enter values in order.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key ‘18’?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Main.cpp

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo.h"


template <typename T>
Node<T> *buildLinkedList(T arr[], int size) {
    Node<T> *head = new_node<T>(arr[0]);
    Node<T> *tail = head;
    for (int i = 1; i < size; i++) {
        tail->next = new_node<T>(arr[i]);
        tail = tail->next;
    }
    return head;
}

int main() {
    // ============================================================
    // PROBLEM 1 & 2 DATA:
    // ============================================================
    int list1[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size1 = 10;
    Node<int> *head1 = buildLinkedList(list1, size1);

    std::cout << "============================================================\n";
    std::cout << "PROBLEM 1: - 18\n";
    std::cout << "List: [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]\n";
    std::cout << "============================================================\n";
    linearSearchArr<int>(list1, size1, 18);
    linearSearchLL<int>(head1, 18);

    std::cout << "\n============================================================\n";
    std::cout << "PROBLEM 2: - 18\n";
    std::cout << "============================================================\n";
    countOccurrencesArr<int>(list1, size1, 18);
    countOccurrencesLL<int>(head1, 18);

    // check:
    std::cout << "\n-- Extra check on other repeating values --\n";
    countOccurrencesArr<int>(list1, size1, 19);
    countOccurrencesArr<int>(list1, size1, 14);
```

```cpp
    std::cout << "\n============================================================\n";
    std::cout << "PROBLEM 3: Binary search - 8\n";
    std::cout << "List: [3, 5, 6, 8, 11, 12, 14, 15, 17, 18]\n";
    std::cout << "============================================================\n";
    binarySearchArr<int>(list2, size2, 8);

    std::cout << "\n============================================================\n";
    std::cout << "PROBLEM 4: Binary search (RECURSIVE) for key - 8\n";
    std::cout << "============================================================\n";
    binarySearchRecursiveArr<int>(list2, 0, size2 - 1, 8);

    return 0;
}
```

Searchingalgo.h

```h
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

#include "node.h"
#include <iostream>

// ============================================================
// PROBLEM 1: Sequential (linear) search - array version
// Returns the index found (or -1) and prints how many comparisons it took.
// ============================================================
template <typename T>
int linearSearchArr(T data[], int size, T dataFind) {
    int comparisons = 0;
    for (int i = 0; i < size; i++) {
        comparisons++;
        if (data[i] == dataFind) {
            std::cout << "[Array] Found " << dataFind << " at index " << i
                      << " -> took " << comparisons << " comparison(s).\n";
            return i;
        }
    }
    std::cout << "[Array] " << dataFind << " not found -> took "
              << comparisons << " comparison(s).\n";
    return -1;
}

// ============================================================
// PROBLEM 1: Sequential (linear) search
// ============================================================
template <typename T>
void linearSearchLL(Node<T> *node, T dataFind) {
    int comparisons = 0;

    while (node != nullptr) {
        comparisons++;
        // CREATE AN IF CONDITION THAT WILL COMPARE THE SEARCH ITEM TO THE CURRENT NODE DATA
        if (node->data == dataFind) {
            std::cout << "[Linked List] Found " << dataFind << " -> took "
                      << comparisons << " comparison(s).\n";
            return;
        }
        // ITERATOR / MOVING OF NODE
        node = node->next;
    }
    // DISPLAY IF UNSUCCESSFUL
    std::cout << "[Linked List] " << dataFind << " not found -> took "
              << comparisons << " comparison(s).\n";
}

// ============================================================
// PROBLEM 2: Modified sequential search
// ============================================================
template <typename T>
int countOccurrencesArr(T data[], int size, T dataFind) {
    int comparisons = 0;
    int occurrences = 0;
```

```
    for (int i = 0; i < size; i++) {
        comparisons++;
        if (data[i] == dataFind) {
            occurrences++;
        }
    }

    std::cout << "[Array] " << dataFind << " appears " << occurrences
              << " time(s) -> took " << comparisons << " comparison(s) total.\n";
    return occurrences;
}

// ================================================================
// PROBLEM 2: Modified sequential search
// ================================================================
template <typename T>
int countOccurrencesLL(Node<T> *node, T dataFind) {
    int comparisons = 0;
    int occurrences = 0;

    while (node != nullptr) {
        comparisons++;
        if (node->data == dataFind) {
            occurrences++;
        }
        node = node->next;
    }

    std::cout << "[Linked List] " << dataFind << " appears " << occurrences
              << " time(s) -> took " << comparisons << " comparison(s) total.\n";
    return occurrences;
}

// ================================================================
// PROBLEM 3: Binary search (ITERATIVE)
// ================================================================
template <typename T>
int binarySearchArr(T data[], int size, T dataFind) {
    int low = 0, high = size - 1;
    int iteration = 0;

    while (low <= high) {
        iteration++;
        int mid = low + (high - low) / 2;

        std::cout << "Iteration " << iteration << ": low=" << low
                  << " high=" << high << " mid=" << mid
                  << " data[mid]=" << data[mid] << "\n";

        if (dataFind == data[mid]) {
            std::cout << "Searching is success! Found " << dataFind
                      << " at index " << mid << " in " << iteration
```

Xx1

```
                << " high=" << high << " mid=" << mid
                << " data[mid]=" << data[mid] << "\n";

        if (dataFind == data[mid]) {
            std::cout << "Searching is success! Found " << dataFind
                      << " at index " << mid << " in " << iteration
                      << " iteration(s).\n";
            return mid;
        } else if (dataFind > data[mid]) {
            low = mid + 1;   // discard left half
        } else {
            high = mid - 1;  // discard right half
        }
    }

    std::cout << "The search is unsuccessful. (" << iteration << " iteration(s))\n";
    return -1;
}

// ========================================================================
// PROBLEM 4: Binary search (RECURSIVE)
// ========================================================================
template <typename T>
int binarySearchRecursiveArr(T data[], int low, int high, T dataFind, int iteration = 1) {
    if (low > high) {
        std::cout << "The search is unsuccessful. (" << iteration - 1 << " iteration(s))\n";
        return -1;
    }

    int mid = low + (high - low) / 2;

    std::cout << "Iteration " << iteration << ": low=" << low
        << " high=" << high << " mid=" << mid
        << " data[mid]=" << data[mid] << "\n";

    if (dataFind == data[mid]) {
        std::cout << "Searching is success! Found "
        << dataFind << " at index " << mid << " in " << iteration
        << " iteration(s).\n";
        return mid;
    } else if (dataFind > data[mid]) {
        return binarySearchRecursiveArr(data, mid + 1, high, dataFind, iteration + 1);
    } else {
        return binarySearchRecursiveArr(data, low, mid - 1, dataFind, iteration + 1);
    }
}

#endif
```

Node.h

```
C node.h > ...
1   #ifndef NODE_H
2   #define NODE_H
3
4   template <typename T>
5   class Node {
6   public:
7       T data;
8       Node<T> *next;
9   };
10
11  template <typename T>
12  Node<T> *new_node(T newData) {
13      Node<T> *newNode = new Node<T>;
14      newNode->data = newData;
15      newNode->next = nullptr;
16      return newNode;
17  }
18
19  #endif
```

Console output

```
============================================================
PROBLEM 1: = 18
List: [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]
============================================================
[Array] Found 18 at index 1 -> took 2 comparison(s).
[Linked List] Found 18 -> took 2 comparison(s).

============================================================
PROBLEM 2: = 18
============================================================
```

```
============================================================
PROBLEM 2: = 18
============================================================
[Array] 18 appears 2 time(s) -> took 10 comparison(s) total.
[Linked List] 18 appears 2 time(s) -> took 10 comparison(s) tota

-- Extra check on other repeating values --
[Array] 19 appears 2 time(s) -> took 10 comparison(s) total.
[Array] 14 appears 2 time(s) -> took 10 comparison(s) total.

============================================================
```

```
==========================================================
PROBLEM 3: Binary search = 8
List: [3, 5, 6, 8, 11, 12, 14, 15, 17, 18]
==========================================================
Iteration 1: low=0 high=9 mid=4 data[mid]=11
Iteration 2: low=0 high=3 mid=1 data[mid]=5
Iteration 3: low=2 high=3 mid=2 data[mid]=6
Iteration 4: low=3 high=3 mid=3 data[mid]=8
Searching is success! Found 8 at index 3 in 4 iteration(s).

==========================================================
PROBLEM 4: Binary search (RECURSIVE) for key = 8
==========================================================
Iteration 1: low=0 high=9 mid=4 data[mid]=11
Iteration 2: low=0 high=3 mid=1 data[mid]=5
Iteration 3: low=2 high=3 mid=2 data[mid]=6
Iteration 4: low=3 high=3 mid=3 data[mid]=8
Searching is success! Found 8 at index 3 in 4 iteration(s).
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\HOA6\supplementary>
```

Conclusion:

This activity clarified the trade-offs between sequential and binary search, and how data structure choice (array vs. linked list) affects implementation but not always outcome. Linear search checks elements one by one and works on any data regardless of order, but scales poorly with list size — as seen when counting repeated occurrences forced a full scan every time. Binary search is far more efficient (finding 8 in just 4 iterations instead of up to 10 comparisons) but only works correctly on sorted data, a requirement Table 6-3b's out-of-order input (12, 10) accidentally violated. Implementing binary search on a linked list also showed that the same O(log n) iteration count doesn't mean equal effort — without random access, the getMiddle helper has to physically walk to the midpoint each time, making it less efficient in practice than the array version. Comparing the iterative and recursive binary search traces further showed that recursion wasn't a different algorithm, just a different way of expressing the same logic.

Overall, I think I handled the core concepts well, correctly identifying comparison counts and iteration behavior across all six tables. My main area for improvement is being more disciplined about validating preconditions — binary search doesn't check that its input is sorted, so that responsibility falls on me as the programmer, and I should build in that safeguard rather than assume the data is correctly ordered. I'd also like to get more comfortable tracing recursive calls by hand, since it took more effort than following the equivalent iterative loop

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.